

Blaze: An Efficient Bit-Sparse Attention Architecture With Workload Orchestration Optimization

Runzhou Zhang*, Faxian Sun, Yiming Wang, Kunchen Zou, Zhinan Qin, Jianli Chen, Jun Yu and Kun Wang[†]
State Key Lab of Integrated Chips & Systems, and School of Microelectronics, Fudan University, Shanghai, China
Email: *23212020028@m.fudan.edu.cn, [†]kun.wang@ieee.org

Abstract—The attention mechanism is a core neural network primitive widely utilized in state-of-the-art models of Natural Language Processing (NLP) applications. However, the high computational complexity and substantial power consumption hinder its deployment and efficient inference. To address these challenges, various methods leveraging sparsity and quantization have been proposed. Compared to these methods, the exploitation of abundant bit-level sparsity in attention-based models presents great potential for the performance enhancement of attention inference. Existing bit-sparsity optimization methods primarily focus on Convolutional Neural Networks (CNNs), which are not ideally suitable for the attention mechanism, and they have not effectively solved the workload imbalance and hardware under-utilization issues caused by the irregular distribution of non-zero bits in tensor data. In this work, we introduce **Blaze**, an efficient attention architecture that leverages both value and bit-level sparsity in tensor data along with workload orchestration optimization. To mitigate the workload imbalance issues often encountered by sparse bit-serial architecture, we propose an Approximate-Computing-Based (ACB) workload orchestration mechanism. Additionally, to fully exploit the redundancy in the attention mechanism, we propose a Leading-Booth mechanism to further enhance the performance of attention computation. We also design a reconfigurable computing engine to support both mechanisms. Experimental results indicate that, compared to state-of-the-art (SOTA) attention accelerators, our **Blaze** can achieve $2.37\times \sim 6.18\times$ improvement in performance and $9.69\times \sim 43.96\times$ enhancement in energy efficiency. Our accelerator can reach up to $1.58\times$ speedup in attention computing performance compared with the SOTA bit-sparse accelerator.

I. INTRODUCTION

Recently, the attention mechanism [1] has revolutionized the field of Artificial Intelligence (AI), leading to the development of advanced models such as Transformer [1], BERT [2], and GPT-2 [3] that have demonstrated unparalleled performance across a wide array of tasks. However, the deployment of attention-based models is often hindered by their high power consumption and substantial computational complexity. As the demand for attention-based model deployment expands across various applications, the pursuit of energy-efficient attention inference architectures has become increasingly critical. This necessity fuels the search for innovative approaches that can enhance both performance and energy efficiency in attention accelerators.

To improve the inference efficiency of attention-based models, various accelerators utilizing sparsity and quantization have been proposed. SpAtten [4] implements token pruning to leverage the sparsity present in sentences. Sanger [5] utilizes dynamic structured sparsity for attention acceleration. DTQAtten [6] and DTATrans [7] dynamically quantize tokens using mixed precision, while DEQ [8] further proposes the dynamic quantization of elements with mixed precision. Although these sparsity and quantization methods have significantly enhanced the inference efficiency of the attention mechanism, they primarily focus on value-level optimizations, overlooking the potential for bit-level enhancements. For non-zero values, not all bits are effective, which motivates us to exploit bit-level sparsity to improve the performance of attention inference.

Sparse bit-serial architectures are widely employed to exploit bit-level sparsity. Rather than processing operands directly, these

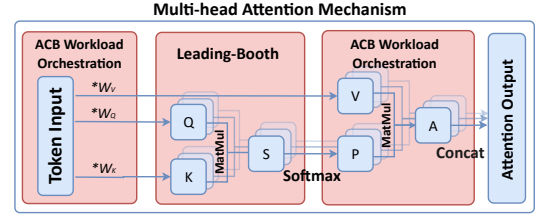


Fig. 1. The application of ACB workload orchestration and Leading-Booth computing schemes in the attention mechanism.

architectures treat integer operands as sequences of bits to enable the skipping of zero bits [9]–[11]. Specifically, Bit-Pragmatic [9] serializes activations while keeping weights fixed within the bit-serial architecture. Laconic [10] utilizes bit sparsity in both weights and activations. AdaS [11] selectively serializes the side with the less effective Booth term. However, these sparse bit-serial accelerators face workload imbalance and hardware under-utilization due to the irregular distribution of non-zero bits in activations and weights. This inconsistency results in some processing elements (PEs) being active while others remain idle. Although AdaS [11] partially mitigates this issue through group-wise bi-directional inner-join for workload orchestration within the same PE, the imbalance across the entire PE array remains unresolved.

To alleviate the workload imbalance problem, Bit-Balance [12] proposes a channel-wise bit-sparsity quantization method, while BitCluster [13] introduces a hardware-friendly quantization approach to ensure that each scalar element in the weight tensor has the same number of effective bits. Both methods tackle the imbalance by serializing weights and equalizing effective bits through software processing. These methods are applicable to Convolutional Neural Networks (CNNs) and the linear layers in Transformer models, as the weights of linear layers can be predetermined before inference. However, these approaches are less suitable for the attention mechanism, where the Queries (Qs), Keys (Ks), and Values (Vs) in the attention are generated at run-time. Therefore, it is necessary to dynamically schedule the workload at the hardware level. BSViT [14] aims to accelerate ViT [15] through a weight bit-group quantization scheme and a dense-sparse hybrid data flow for workload orchestration among PEs. However, in terms of exploiting the inherent characteristics of Transformers, while BSViT employs a dynamic patch quantization strategy for input data processing, it overlooks the architectural and acceleration potential within the attention mechanism.

To address the aforementioned challenges, we propose **Blaze**, an efficient bit-sparse attention architecture, featuring a dedicated optimization scheme for workload orchestration optimization. The contributions of this work are as follows:

- We propose a dedicated sparse bit-serial hardware architecture, **Blaze**, to fully exploit both the value and bit-level sparsity for attention acceleration. We design a reconfigurable computing array to support our optimization schemes, including the Leading-Booth and the Approximate-Computing-Based workload orchestration mechanisms.

[†]Corresponding author

- We present an Approximate-Computing-Based (ACB) workload orchestration mechanism to tackle the workload imbalance problem by skipping certain least significant Booth terms in a PE line. This approach achieves workload orchestration across PEs within the line, as well as bit-serial multipliers inside each PE.
- To utilize the abundant redundancy in attention, we introduce the Leading-Booth computing mechanism in the attention computation to replace full-precision calculations. By computing only the first non-zero Booth term, the computational workload is significantly reduced while maintaining the accuracy.
- Experimental results demonstrate that our Leading-Booth and ACB dynamic workload orchestration mechanisms incur at most a 0.9% accuracy loss. Compared with the SOTA attention accelerators, our design can achieve up to $2.37\times \sim 6.18\times$ improvement in performance and $9.69\times \sim 43.96\times$ enhancement in energy efficiency. Meanwhile, our design can reach up to $1.58\times$ speedup in attention computing performance compared with the SOTA bit-sparse accelerator.

II. BACKGROUND AND MOTIVATIONS

A. Attention Mechanism

As shown in Fig. 1, the attention mechanism consists of multiple linear projections and non-linear activation layers. Initially, input features are projected onto the representations Q , K , and V through three linear transformations. Attention scores are then computed by performing matrix multiplication $Q \cdot K^T$. After scaling, these attention scores (S) are passed to a Softmax function to obtain attention probabilities P . Finally, P is multiplied by V to produce the final output matrix, which represents the weighted combination of V vectors based on the relevance determined by Q and K pairs.

B. Bit-Serial Architecture

Bit-sparse accelerators typically employ a bit-serial architecture to leverage bit-level sparsity. The bit-serial computation paradigm can be denoted as follows:

$$A \times B = \sum A \times b_i \times 2^i = \sum (A \times b_i) \ll i, \quad (1)$$

where A is fixed and b_i is each bit of B . Basic bit-serial multiplication can be optimized through zero-bit skipping, which leads to the derivation of sparse bit-serial multiplication. However, the irregular distribution of effective bits often results in workload imbalances among the PEs within the PE array, ultimately diminishing the performance of sparse bit-serial computation.

C. Motivation

Our work is inspired by the following considerations. First, bit-level sparsity is abundant in attention-based models such as Transformers. As illustrated in Table I, bit-level sparsity is abundant in the attention computation stage of the BERT [2] models, which is on average $2.28\times$ times over the value sparsity in Table I. This finding motivates us to develop architectures that utilize bit-level sparsity in the attention mechanism. Secondly, as Sanger [5] has demonstrated, the inherent redundancy in the attention mechanism is exceedingly large. For example, Sanger can even achieve 92% attention sparsity without incurring accuracy loss. This insight motivates us to develop aggressive bit-level optimization techniques to exploit the inherent redundancy in attention computation. Finally, unlike the weights that can be predetermined before inference, the Q , K , and V matrices are generated at run-time. Therefore, dynamic mitigation schemes at the hardware level are more suitable for the workload orchestration of attention computation.

Motivated by the above considerations, we propose a sparse bit-serial architecture to leverage the abundant value and bit-level sparsity

TABLE I
VALUE AND BIT-LEVEL SPARSITY OF BERT-BASE MODEL ON DIFFERENT NLP DATASETS.

Dataset/Tensor		Weight	Embed	QKV	P
VS/BS percentage (%)	CoLA	2.7/ 49.8	16.3/ 52.3	2.7/ 50.5	93.2/ 98.3
	MRPC	2.4/ 49.4	7.5/ 50.1	2.6/ 50.5	82.6/ 96.9
	QNLI	3.6/ 51.2	18.1/ 53.3	3.5/ 51.3	93.4/ 98.3
	RTE	3.2/ 51.3	14.8/ 54.7	2.9/ 50.6	82.5/ 96.8
	SST-2	4.4/ 53.4	17.8/ 53.3	2.7/ 50.5	92.8/ 98.3
	STS-B	3.3/ 48.8	8.3/ 49.0	2.6/ 50.5	85.4/ 97.1
	SQuAD	2.6/ 52.3	6.5/ 50.5	2.6/ 50.8	86.0/ 98.0
	CLOTH	3.2/ 49.9	6.0/ 53.8	2.7/ 50.5	82.0/ 97.5

in attention computation. As shown in Fig. 1, to further exploit the substantial redundancy in attention, we have developed the Leading-Booth computing mechanism for $Q \cdot K^T$. Finally, we propose the ACB workload orchestration mechanism for QKV generation and $P \cdot V$ stages to dynamically address the issue of workload imbalance.

III. OPTIMIZATION SCHEME

A. Efficient Sparse Bit-Serial Multiplication

To harness the bit sparsity inherent in attention, we adopt the sparse bit-serial computing paradigm, which can skip zero partial products. We prefer Adaptive One-sided Bit-serial Multipliers (AOBMs) compared to basic ones because it can lead to better computational performance by serializing the multiplicand with less effective bits [11]. As for the encoding method to extract non-zero term, we choose Booth encoding because it allows for a more natural processing for signed values, and it can figure out more bit sparsity messages in scalar data [10]. The computation of Booth-encoding-based adaptive one-sided bit-serial processing can be denoted as follows:

$$A_1 \times A_2 = \begin{cases} (\sum_{i=0}^{N_1-1} C_{1i} \times 2^{S_{1i}}) \times A_2 & N_1 \leq N_2 \\ A_1 \times (\sum_{i=0}^{N_2-1} C_{2i} \times 2^{S_{2i}}) & N_1 > N_2, \end{cases} \quad (2)$$

where C , S , and N refer to non-zero Booth terms, their corresponding shift, and Number of non-zero (Nnz) Booth codes, respectively. The hardware structure of AOBM will be introduced in Section IV-B.

B. Overview of ACB Workload Orchestration Mechanism

The sparse bit-serial multiplication described above focuses on the scalar multiplication (SM) level. To facilitate matrix multiplication (MM) tasks, we need to design a PE array with each PE consisting of multiple AOBMs. Due to the irregular distribution of non-zero Booth terms, the AOBMs exhibit different working cycles, resulting in intra-PE workload imbalances. In our design, PEs within the same row of the array share the same data-fetching state; however, the differing working cycles among same-row PEs lead to inter-PE workload imbalance issues.

To mitigate both the inter-PE and intra-PE workload imbalances, it is essential to synchronize the workloads (WLs) across all PEs and AOBMs. Given that attention-based models are robust and exhibit high tolerance to noise, we propose an Approximate-Computation-Based (ACB) workload orchestration mechanism for the sparse bit-serial PE array structure, in order to achieve inter-PE and intra-PE workload orchestration, as well as the reduction of computation workload. The general idea is that each vector multiplication (VM) task consists of multiple SMs, and approximations can be applied to the last non-zero Booth term of the scalar element to adjust the workload at the SM computation level. The approximate computation mode of scalar multiplication can be illustrated as follows:

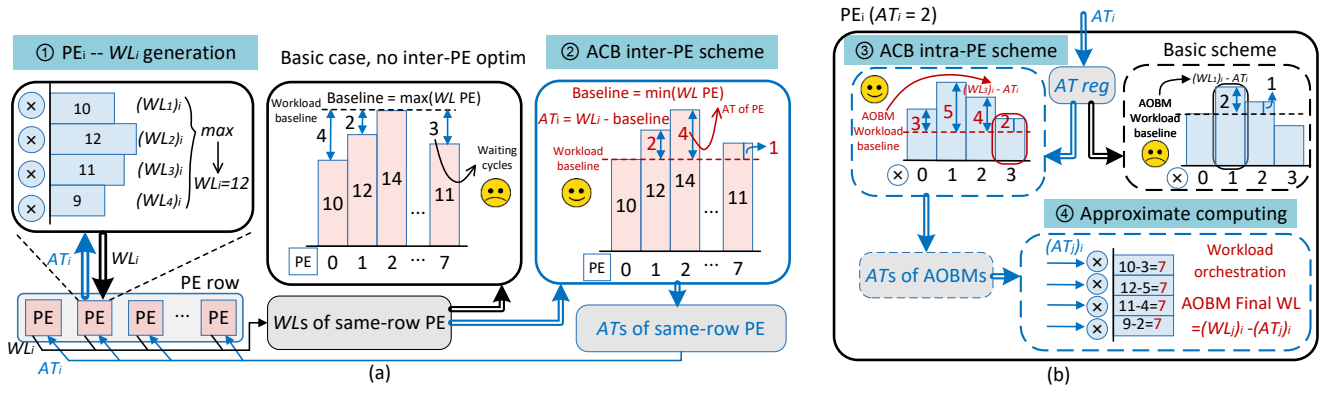


Fig. 2. ACB Workload orchestration analysis, with eight PEs in each row and four AOBMs in each PE. (a) Inter-PE workload orchestration analysis; (b) Intra-PE workload orchestration analysis.

$$A_1 \times A_2 = \begin{cases} \left(\sum_{i=0}^{N_1-2} C_{1i} \times 2^{S_{1i}} \right) \times A_2 & N_1 \leq N_2, N_1 > 1 \\ A_1 \times \left(\sum_{i=0}^{N_2-2} C_{2i} \times 2^{S_{2i}} \right) & N_1 > N_2, N_2 > 1, \end{cases} \quad (3)$$

where the least significant bit (LSB) Booth term is skipped if the operand has more than two non-zero Booth terms.

C. Optimization for Inter-PE Workload Orchestration

The ACB workload orchestration mechanism comprises four stages: ① the WL generation, ② the inter-PE and ③ intra-PE Approximation Threshold (AT) generation, and ④ the approximate computing stage. The definition of AT means the quantities of approximate computation that should be introduced, which is equal to the difference between the original and optimized WL . The ④ approximate computing stage will be introduced in Section IV-D. At ① the WL generation stage, the WL of each adaptive one-sided bit-serial multiplier and PE unit is determined by the Nnz data of the processed vector as follows:

$$\begin{cases} (S_{tile})_i = (NnzA)_{tile} \leq ((NnzB)_{tile})_i \\ (WL_j)_i = (S_{tile_j})_i \cdot (NnzA)_{tile_j} + (S_{tile_j})_i \cdot ((NnzB)_{tile_j})_i \\ WL_i = \max((WL_1)_i, (WL_2)_i, \dots, (WL_{N_{Mult}})_i) \\ i = 1, \dots, N_{col}, j = 1, \dots, N_{Mult}, \end{cases} \quad (4)$$

with WL of PE determined by the maximum WL of AOBMs. Then the AT of each PE is obtained at ② inter-PE AT generation stage.

Fig. 2 (a) shows the workload orchestration analysis and optimization scheme of the ACB mechanism at the inter-PE level. In the basic case, after all the VM data are prepared, the same-row PEs must wait for the slowest PE which has the maximum WL to complete. Here, we define the orchestrated working cycles as the workload baseline, which is suitable for the description at both PE and AOBM levels. As shown in the ACB inter-PE scheme of Fig. 2 (a), we consider adjusting the workload baseline to the minimum WL and discarding the least significant bit (LSB) non-zero Booth terms in the other heavy-workload VM tasks. The AT of PE is defined as:

$$\begin{cases} WL_{PE_baseline} = \min(WL_1, WL_2, \dots, WL_{N_{col}}) \\ AT_i = WL_i - WL_{PE_baseline}, i = 1, \dots, N_{col}, \end{cases} \quad (5)$$

where AT is equal to the difference between each WL of PE and workload baseline which is adjusted to the minimum WL in the PE row. After ② the inter-PE AT generation, the analysis of ③ intra-PE AT generation will be introduced in the following subsection.

D. Optimization for Intra-PE Workload Orchestration

After each PE receives a VM task, each AOBM in the same PE is allocated with a tiled VM task, and the different workloads

among the tiled VM tasks cause the intra-PE workload imbalance. For ③ the intra-PE workload orchestration analysis, we decide to apply approximate computing to the AOBM level. The key issue is to determine the AT of each AOBM in one single PE according to the current data, including the AT of PE and WL of each AOBM.

The basic approach shown in Fig. 2 (b) allocates the AT of PE to the AOBM with maximum WL , making the AT of this AOBM equal to that of the PE. Consequently, the workload baseline at AOBM level is calculated by subtracting the AT of PE from that maximum- WL value. Although this approach reduces workload imbalance, further optimizations can leverage the error resilience of attention-based models. Like the analysis conducted for the inter-PE case, intra-PE workload orchestration aims to find new orchestrated working cycles for all AOBMs in one PE. As depicted in Fig. 2 (b), compared to the basic approach, we choose to allocate the AT of PE to the AOBM with the minimum WL instead of the maximum- WL one, allowing the AOBM workload baseline to be determined by the difference between the minimum AOBM WL and the PE's AT . The AOBM workload baseline can be formalized as follows:

$$\begin{cases} WL_{AOBM_baseline} = \min((WL_1)_i, (WL_2)_i, \dots, (WL_{N_{Mult}})_i) \\ (AT_j)_i = AT_i - WL_{AOBM_baseline} + (WL_j)_i \\ i = 1, \dots, N_{col}, j = 1, \dots, N_{Mult}. \end{cases} \quad (6)$$

Consequently, the AT s for each AOBM can be derived through basic operations between $Nnzs$ and WL s. The AT of each AOBM finally determines the quantities of approximate computation introduced in each tiled VM task, as well as the workload optimization quantity of each AOBM in one PE. As a result, the ACB mechanism enables the PE array to achieve both inter-PE and intra-PE workload orchestration while significantly reducing computation.

E. Leading-Booth Computing Mechanism

There is abundant redundancy in the attention mechanism, thereby allowing for aggressive optimization methods with minor accuracy loss. To leverage this characteristic, we introduce the Leading-Booth computing mechanism into attention computation. Based on the adaptive one-sided paradigm of sparse bit-serial multiplication, the Leading-Booth mechanism retains only the first non-zero Booth term of the value used for serialization, thus providing greater workload mitigation than the ACB mechanism. The computing process of Leading-Booth mechanism can be formalized as follows:

$$A_1 \times A_2 = \begin{cases} (C_1 \times 2^{S_{1i}}) \times A_2 & N_1 \leq N_2 \\ A_1 \times (C_2 \times 2^{S_{2i}}) & N_1 > N_2, \end{cases} \quad (7)$$

where only the first non-zero Booth term (C_1 or C_2) is remained in bit-serial multiplication. This approach simplifies the original adaptive bit-serial multiplication into a single-cycle operation.

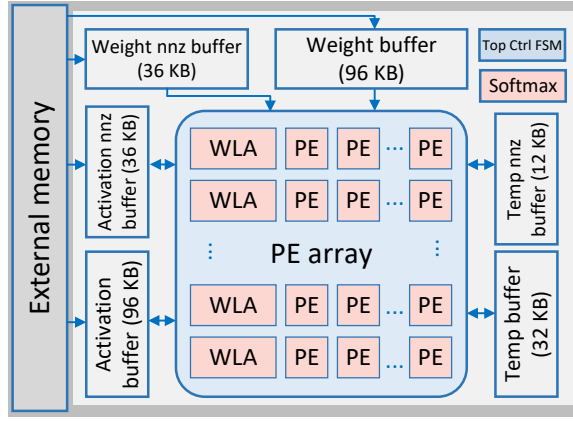


Fig. 3. The overall architecture of **Blaze**. The light gray part represents our memory design and corresponding size. The red part shows our computing modules and workload analyzers, as well as the Softmax unit.

In DEQ [8], recomputation is necessary after the low-complexity computation of $Q \cdot K^T$ to mitigate accuracy loss. In contrast, the Leading-Booth mechanism retains more operand information, thereby eliminating the need for recomputation. To ensure accuracy preservation, we apply the Leading-Booth mechanism to $Q \cdot K^T$ while still employing the ACB mechanism for the $P \cdot V$ and QKV generation stages. As will be presented in Section V-B, the Leading-Booth mechanism incurs only a negligible loss of accuracy. Our Leading-Booth method is integrated into the PE through a reconfigurable bit-serial computing core (Subsection IV-D), without the need for extra computing engines. As a result, the Leading-Booth mechanism achieves a hardware-friendly computing solution that balances complexity and accuracy effectively.

IV. BLAZE ARCHITECTURE

A. Overall Architecture

The overall architecture of **Blaze** is illustrated in Fig. 3. Before the inference phase, all embedding data are transferred from the host to the hardware platform. The processing element (PE) array consists of sparse bit-serial PEs and Workload Analyzers (WLAs), with data being transferred from the activation and weight buffers. The output data from matrix multiplication (MM) tasks are stored in temporary buffers. The WLA requires Nnz data for dynamically orchestrated workload assignments, which are computed concurrently alongside activation and weight data for the MM task. Consequently, the initial Nnz values for the embedding and weight data are obtained from the software side, while subsequent Nnz values are generated on the hardware side. The overall data flow in the **Blaze** architecture is managed by a top-level controlling Finite-State Machine (FSM).

B. Sparse Bit-Serial PE

In our design, each PE comprises four parallel AOBMs, allowing simultaneous processing within one single cycle and achieving an optimal balance between performance and hardware efficiency. The structure of the sparse bit-serial PE is illustrated in Fig. 4. Each AOBM inside the single PE handles one quarter of a tiled vector multiplication (VM) task, which consists of a value fetcher, an adaptive encoder, a sparse bit-serial core, and a controller for the ACB workload orchestration mechanism.

In our design, each 8-bit operand in the data vector is assigned a corresponding 3-bit input Nnz and a generated 1-bit mask, indicating bit-level and value sparsity workloads, respectively. A bitwise AND operation on the value bitmask vectors allows AOBMs to selectively

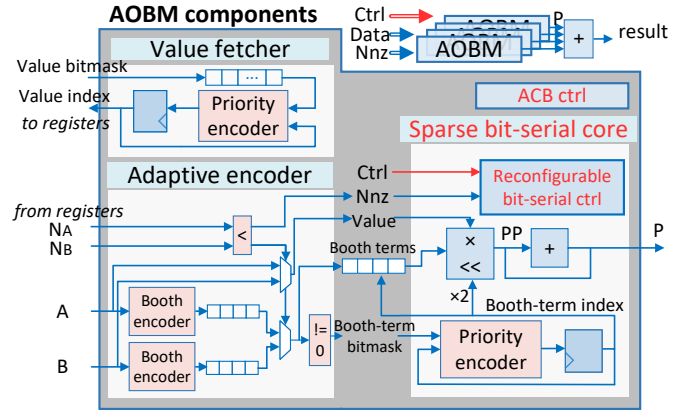


Fig. 4. Structure of the sparse bit-serial PE with four AOBMs.

fetch non-zero values via a priority encoder, using the previous output as the mask. This method efficiently obtains non-zero value indices while skipping zeros, avoiding unnecessary computations [16]. Similarly, zero-value Booth terms are excluded using Booth-term bitmask signals, directing the sparse bit-serial core to retrieve non-zero Booth terms through the priority encoder. Consequently, each AOBM's computing process is optimized by leveraging both value and bit-level sparsity in attention-based models, thereby significantly enhancing computational efficiency. To support ACB workload orchestration and Leading-Booth computing modes, we incorporate a reconfigurable sparse bit-serial core within the PE, detailed in Section IV-D. The entire PE operation is executed in a pipelined manner.

C. Workload Analyzer

As described in Section III-C, prior to the implementation of approximate computing, it is essential to acquire the workload (WL) and Approximate Threshold (AT) at both the PE and AOBM levels. The workload analyzer structure in Fig. 5 details the hardware computing process of WL and AT generation. The tile counter records the number of processed operands pairs for one single tiled VM task. The WL of each AOBM within a single PE can be obtained through the operations between tiled Nnz vectors. After the obtainment of WL , the AT at PE and AOBM levels can be computed via further operations introduced in Sections III-C and III-D.

D. Reconfigurable Bit-Serial Computing Core

Once the AT for each AOBM is obtained, these data, along with the corresponding control signals, are transferred to the AOBMs within each PE. As depicted in Fig. 4, the workload-orchestrated sparse bit-serial computing process is managed by the ACB controller and the reconfigurable bit-serial controller in the AOBM, both functioning as counters to regulate the amount of approximate computation at the levels of tiled VM task and fundamental scalar multiplication (SM), respectively. The ACB controller tracks the amount of approximate computation at the tiled VM task level. For each AOBM, with the input AT set as counter threshold, the ACB controller can decide the assignment of approximate computation for specific pairs of scalar operands. When the counter of ACB controller reaches the AT of the AOBM, or when all non-zero values in the current vector tile have been processed, no further approximate computation will be introduced into subsequent SM computations.

The sparse bit-serial controller can switch between the ACB and Leading-Booth computing modes based on the input control signals. This controller operates as a reconfigurable counter. In the Leading-Booth computing mode, the counter threshold is set to one since only the leading Booth term is needed. As illustrated in Fig. 4,

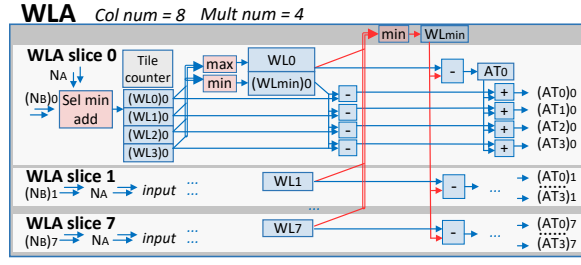


Fig. 5. The WLA structure, which serves to generate WL s and AT s according to the ACB workload orchestration mechanism.

in the ACB computing mode, the Nnz output from the adaptive encoder serves as the initial counter threshold. As introduced in Section III-B, in our approximate computing mechanism, the last Booth term from the adaptive encoder output shown in Fig. 4 is discarded, resulting in an extra reduction of the initial counter threshold by one. To prevent significant computational discrepancies and accuracy loss, we stipulate that if the Nnz output of the adaptive encoder in the AOBM is equal to one, approximate computing will not be applied to that particular operand pair. Consequently, the reconfigurable bit-serial computing core efficiently implements both Leading-Booth computing and the approximate computing stage of the ACB mechanism within our **Blaze** architecture.

V. EVALUATION

A. Evaluation Setup

Benchmarks. To validate the computing performance of our bit-level optimization schemes, we select the Bert-base model and fine-tune it on the SQuAD [17], CLOTH [18], and six GLUE [19] text classification tasks. The evaluation metrics are Matthews correlation for CoLA, F1 for MRPC, accuracy for QNLI, RTE, SST-2 and CLOTH, Pearson correlation for STSB, and exact-match for SQuAD. We have developed our coding experiments based on PyTorch [20] and Huggingface [21]. We fine-tune the pretrained model for 5 epochs using the AdamW optimizer. For the GLUE, SQuAD and CLOTH datasets, the maximum sequence length for BERT is set to 128.

Hardware implementation. We implement the **Blaze** architecture in Verilog HDL and synthesize it using the Synopsys Design Compiler at 500MHz under TSMC 28nm HPC+ technology. The power consumption of SRAM-based on-chip buffers is obtained by CACTI 7 [22]. Fig. 6 provides the detailed area and power breakdown of **Blaze**. Our PE array is configured as 8×8 with four AOBMs placed in each PE. Since each 8-bit value can be encoded into four Booth terms and each AOBM tackles one Booth-term multiplication in a single cycle, the computing capacity of four AOBMs in each PE is equal to one 8-bit multiplier per PE. The total SRAM capacity is 308KB, which is made up of input activation, weight, temporary data and their corresponding Nnz buffers. Our accelerator has a total area of $0.968mm^2$ and a total power of $387.7mW$. We develop a cycle-accurate simulator for our proposed **Blaze** architecture to collect statistics on computation, buffer usage, and external memory accesses for each attention computation workload. For off-chip traffic, we follow DEQ [8] and assume 20 pJ/bit for energy consumption, with a 256 bits/cycle off-chip bandwidth.

B. Accuracy Evaluation

Table II presents the accuracy results of Sanger [5], DTQAtten [6], DEQ [8], and **Blaze**. We conducted the Leading-Booth mechanism in $Q \cdot K^T$ and replaced the rest matrix multiplication in attention as the ACB workload-orchestrated matrix multiplication. The accuracy results show that **Blaze** achieves -0.08%, -0.58%, 0.70%, 0.03% accuracy improvement on average over baseline, Sanger, DTQAtten,

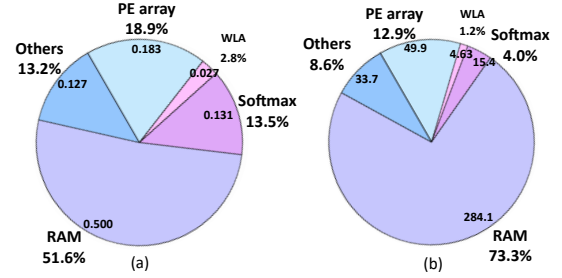


Fig. 6. (a) Area and (b) Power breakdown of **Blaze**.

and DEQ, respectively. Our method incurs at most only 0.9% accuracy degradation in the MRPC dataset, while in the QNLI dataset our methods achieve the highest accuracy results. These accuracy results indicate that, although our approaches are lossy, their impact on the model's accuracy is minimal.

Note that unlike Sanger, DTQAtten, and DEQ, our methods also apply to the linear layers in attention while these designs can only accelerate the $Q \cdot K^T$ and $P \cdot V$ stages. Although introducing more lossy components to attention without recomputation stages, our work still achieves comparable accuracy results compared with DEQ. At the $Q \cdot K^T$ stage, our Leading-Booth mechanism employs only 3-bit multiplication with no recomputation, while DEQ performs 4-bit multiplication and needs recomputation to mitigate accuracy loss. We believe that this difference arises because DEQ utilizes the 4-bit data statically, while our Leading-Booth Mechanism dynamically utilizes the first effective 3-bit (Booth term) data.

C. Performance Comparison with SOTA Attention Accelerators

We have conducted extensive experiments on SQuAD, CLOTH, and six GLUE datasets to compare our accelerator with state-of-the-art (SOTA) attention accelerators. We create cycle-accurate simulators for Sanger, DTQAtten, and DEQ to evaluate their performance statistics considering both algorithmic and hardware profits. For a fair comparison, we restrict the accelerators to the same peak computing capacity and buffer size. Fig. 7 presents performance results normalized to Sanger. Experimental results show that **Blaze** achieves average normalized performance gains of $6.18\times$, $5.13\times$, and $2.37\times$ over Sanger, DTQAtten and DEQ, respectively. Regarding energy efficiency, our accelerator improves efficiency by $43.96\times$ compared to Sanger and $9.69\times$ compared to DTQAtten.

The performance and energy efficiency gains of **Blaze** over Sanger, DTQAtten, and DEQ stem from three key aspects. First, **Blaze** effectively exploits bit-level sparsity. While Sanger, DTQAtten, and DEQ adopt optimization methods for data flow and processing element utilization, they overlook the abundant bit-level sparsity inherent in attention-based models. In contrast, **Blaze** employs an efficient sparse bit-serial PE design. Second, building on the basis of the sparse bit-serial architecture, **Blaze** incorporates the ACB mechanism for workload orchestration, discarding certain least significant Booth terms to optimize computation workloads. Importantly, the ACB mechanism coordinates workloads across the entire PE array at both the inter-PE and intra-PE levels, significantly reducing idle cycles and enhancing computation resource utilization.

Finally, **Blaze** optimizes the computation workload of $Q \cdot K^T$ using the Leading-Booth mechanism. Unlike DEQ, which requires high-precision recomputation to ensure accuracy, the Leading-Booth mechanism for $Q \cdot K^T$ requires no additional recomputation because of its strong information retention capability, thus greatly optimizing the computation workload without sacrificing accuracy. While Sanger and DTQAtten also avoid additional recomputation, their methods involve the removal of certain elements through pruning and the

TABLE II
ACCURACY RESULTS OF DIFFERENT DESIGNS ON NLP DATASETS.

Dataset	CoLA	MRPC	QNLI	RTE
Baseline	55.7	89.8	91.1	63.2
Sanger	57.2	88.7	91.0	66.4
DTQAtten	54.3	89.7	90.5	63.0
DEQ	58.3	89.6	90.8	62.4
Blaze	55.3	88.9	91.5	65.1
Dataset	SST-2	STS-B	SQuaD	CLOTH
Baseline	92.8	88.5	81.3	79.6
Sanger	93.1	88.6	81.4	79.6
DTQAtten	92.2	/	80.5	/
DEQ	92.5	88.4	80.4	78.8
Blaze	92.3	87.9	81.1	79.3

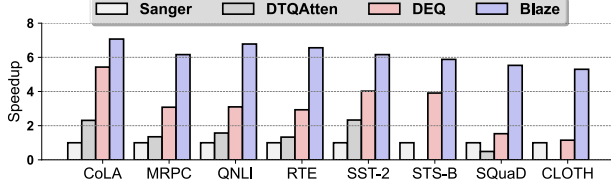


Fig. 7. Speedup of attention accelerators, with the performance of Sanger set as the normalized standard.

application of mixed-precision techniques. These methods, however, introduce a risk of omitting essential information, thus making it challenging to realize substantial performance gains when operating within stringent accuracy requirements.

D. Effectiveness of Workload Orchestration Optimization

We have conducted experiments to compare with the SOTA bit-sparse accelerator and evaluate the effectiveness of our workload-orchestration optimization schemes. We develop a cycle-accurate simulator to collect the performance and MAC utilization statistics of the bit-sparse accelerator AdaS on the SQuaD, CLOTH, and six GLUE datasets. Meanwhile, cycle-accurate simulators are also created for the value-sparsity optimized (VSO) and bit-sparsity optimized (BSO) multiplication mechanisms for performance comparison, which merely skip non-zero operands and non-zero Booth terms, respectively, without workload orchestration or other optimizations. All mechanisms are evaluated with the same peak computing capacity and buffer size. Fig. 8 shows the performance comparison of these computing mechanisms at different matrix multiplication stages in attention, including QKV generation, $Q \cdot K^T$, and $P \cdot V$.

According to the results, our ACB mechanism achieves $3.40\times$, $2.03\times$, and $1.51\times$ performance gains on average on the linear layers of QKV generation compared to VSO, BSO, and AdaS, respectively. Our ACB mechanism performs better in the $P \cdot V$ computation, reaching on average $7.04\times$ and $4.35\times$ performance gains compared to VSO and BSO. This is due to the higher value and bit-level sparsity at the $P \cdot V$ stage, and our ACB mechanism can make use of both kinds of sparsity to improve computing performance. Although our ACB mechanism achieves a closer performance with AdaS at the $P \cdot V$ stage, the $P \cdot V$ stage accounts for a non-dominant fraction of the computation burden among the whole attention computation process. For evaluations on $Q \cdot K^T$, our Leading-Booth mechanism achieves on average $4.00\times$, $2.28\times$, and $1.94\times$ performance gains compared to the VSO, BSO, and AdaS.

In terms of computing mechanisms, the sparse bit-serial accelerator AdaS is lossless. Recognizing that attention-based models exhibit robust performance, we develop a lossy ACB workload orchestration mechanism that enhances performance with minimal accuracy loss. While AdaS employs a group-shuffling bidirectional-innerjoin mech-

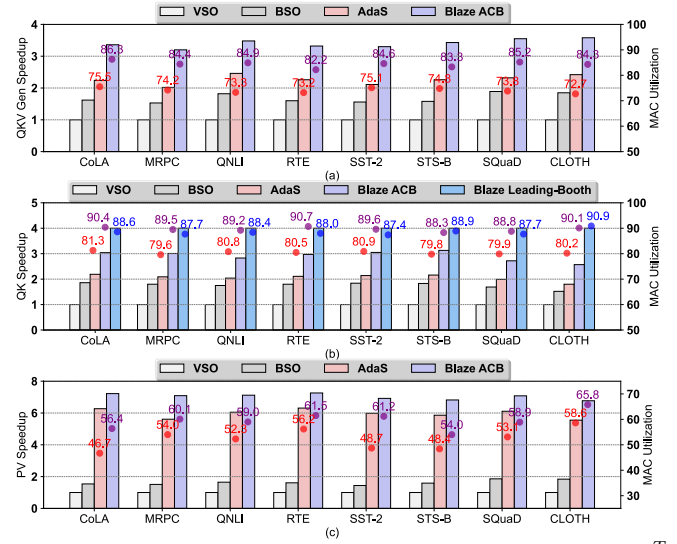


Fig. 8. Speedup and MAC utilization in (a) QKV generation, (b) $Q \cdot K^T$, and (c) $P \cdot V$ stages of different computing mechanisms (AdaS, Blaze ACB and Leading-Booth for MAC utilization), with the performance of VSO set as the normalized performance standard.

anism to address workload imbalance issues [11], it only addresses intra-PE imbalances, leaving the inter-PE imbalances unresolved. Our ACB mechanism effectively alleviates both the intra-PE and inter-PE workload imbalances. As illustrated in Fig. 8, our ACB mechanism achieves average increase by 10.55%, 9.20%, and 7.36% in MAC utilization compared to AdaS during QKV generation, $Q \cdot K^T$, and $P \cdot V$, respectively, thereby showing that the ACB mechanism not only optimizes computation workload, but also achieves more effective workload orchestration and greater computation resource utilization. Additionally, our Leading-Booth mechanism enhances MAC utilization by 8.08% versus AdaS. By leveraging approximate computation, we improve both workload quantities and orchestration, yielding a performance boost of $1.42\times$ to $1.58\times$ compared to AdaS across six GLUE datasets, as well as the SQuaD and CLOTH datasets. Notably, our Leading-Booth mechanism also achieves $1.38\times$ performance gains over the ACB mechanism, as it simplifies the multi-cycle sparse bit-serial multiplication in ACB to a single-cycle process.

VI. CONCLUSION

In this work, we propose **Blaze**, an efficient bit-sparse accelerator for attention-based models. To alleviate the commonly existing load imbalance problem, we propose an Approximate-Computing-Based workload orchestration mechanism. Besides, we develop the Leading-Booth computing mechanism to further utilize the redundancy in attention. The experimental results show that our method incurs only at most a 0.9% accuracy loss compared with the baseline. At the hardware level, our accelerator can achieve $2.37\times \sim 6.18\times$ performance gains, and $9.69\times \sim 43.96\times$ energy efficiency compared with SOTA attention accelerators. Our accelerator can achieve up to $1.58\times$ speedup in attention computing performance compared with the SOTA bit-sparse accelerator AdaS.

ACKNOWLEDGEMENTS

This work was financially supported in part by National Key Research and Development Program of China under Grant 2021YFA1003602, in part by Project of Shanghai EC under Grant 24KXZNA12, and in part by National Natural Science Foundation of China (NSFC) Research Project 92373207. The computations in this research were performed using the CFFF platform of Fudan University.

REFERENCES

- [1] A. Vaswani, "Attention is all you need," *Advances in Neural Information Processing Systems*, 2017.
- [2] J. Devlin, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [4] H. Wang, Z. Zhang, and S. Han, "Spatten: Efficient sparse attention architecture with cascade token and head pruning," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 97–110, IEEE, 2021.
- [5] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, "Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 977–991, 2021.
- [6] T. Yang, D. Li, Z. Song, Y. Zhao, F. Liu, Z. Wang, Z. He, and L. Jiang, "Dtqatten: Leveraging dynamic token-based quantization for efficient attention architecture," in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 700–705, IEEE, 2022.
- [7] T. Yang, F. Ma, X. Li, F. Liu, Y. Zhao, Z. He, and L. Jiang, "Dtatrans: Leveraging dynamic token-based quantization with accuracy compensation mechanism for efficient transformer architecture," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 2, pp. 509–520, 2022.
- [8] X. Wang, Z. Song, Q. Huang, and X. Liang, "Deq: Dynamic element-wise quantization for efficient attention architecture," in *2023 IEEE 41st International Conference on Computer Design (ICCD)*, pp. 623–630, 2023.
- [9] J. Albericio, A. Delmás, P. Judd, S. Sharify, G. O'Leary, R. Genov, and A. Moshovos, "Bit-pragmatic deep neural network computing," in *Proceedings of the 50th annual IEEE/ACM international symposium on microarchitecture*, pp. 382–394, 2017.
- [10] S. Sharify, A. D. Lascorz, M. Mahmoud, M. Nikolic, K. Siu, D. M. Stuart, Z. Poulos, and A. Moshovos, "Laconic deep learning inference acceleration," in *Proceedings of the 46th International Symposium on Computer Architecture*, pp. 304–317, 2019.
- [11] X. Lin, G. Li, Z. Liu, Y. Liu, F. Zhang, Z. Song, N. Jing, and X. Liang, "Adas: A fast and energy-efficient cnn accelerator exploiting bit-sparsity," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, IEEE, 2023.
- [12] W. Sun, Z. Zou, D. Liu, W. Sun, S. Chen, and Y. Kang, "Bit-balance: Model-hardware co-design for accelerating nns by exploiting bit-level sparsity," *IEEE Transactions on Computers*, 2023.
- [13] A. Li, H. Mo, W. Zhu, Q. Li, S. Yin, S. Wei, and L. Liu, "Bitcluster: Fine-grained weight quantization for load-balanced bit-serial neural network accelerators," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 4747–4757, 2022.
- [14] G. Wang, S. Cai, W. Li, D. Lyu, and G. He, "Bsvit: A bit-serial vision transformer accelerator exploiting dynamic patch and weight bit-group quantization," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2024.
- [15] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
- [16] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. Vijaykumar, "Sparten: A sparse tensor accelerator for convolutional neural networks," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 151–165, 2019.
- [17] P. Rajpurkar, "Squad: 100,000+ questions for machine comprehension of text," *arXiv preprint arXiv:1606.05250*, 2016.
- [18] Q. Xie, G. Lai, Z. Dai, and E. Hovy, "Large-scale cloze test dataset created by teachers," *arXiv preprint arXiv:1711.03225*, 2017.
- [19] A. Wang, "Glue: A multi-task benchmark and analysis platform for natural language understanding," *arXiv preprint arXiv:1804.07461*, 2018.
- [20] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.
- [21] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, pp. 38–45, 2020.
- [22] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "Cacti 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 1–25, 2017.